

A Virtual Lab for Learning AI Security and Adversarial Prompt Engineering

Dhanraj Jagadish Devadiga
dhanrajdeeru@gmail.com
Department of Informatics
King's College London
London, United Kingdom

Hannah Cao
hannah.cao@kcl.ac.uk
Department of Informatics
King's College London
London, United Kingdom

Ievgeniia Kuzminykh
ievgeniia.kuzminykh@kcl.ac.uk
Department of Informatics
King's College London
London, United Kingdom

Bogdan Ghita
bogdan.ghita@plymouth.ac.uk
School of Engineering, Computing and Mathematics
University of Plymouth
Plymouth, United Kingdom

Abstract

Computing education increasingly focuses on teaching secure coding and developing secure web applications. Now, with the rise of generative AI, we face new challenges, particularly the potential misuse of this technology for identifying and exploiting software vulnerabilities. This paper presents a virtual learning environment that integrates a secure sandbox with access to multiple commercial and open-source LLMs to support experiential learning in AI security. The lab offers scenario-based exercises that cover web, code, and system-level vulnerabilities. Students craft adversarial prompts, test LLM-generated exploits, and analyse models behaviour using established metrics such as attack success rate (ASR), exploit generation accuracy, and refusal rate. Our expert validation revealed distinct model behaviours: GPT-4o achieved the highest ASR (77.5% in web testing), demonstrating consistent exploit generation, while Claude exhibited the highest safety refusal rate (32.56%). By engaging with these observable outcomes, students develop a critical understanding of LLM capabilities, limitations, and ethical risks. We argue that AI-integrated virtual labs are essential for preparing students to work responsibly with emerging AI-assisted security tools.

CCS Concepts

• **Security and privacy** → **Software and application security**; *Web application security*; • **Computing methodologies** → *Artificial intelligence*; • **Social and professional topics** → *Computing education*.

Keywords

Large Language Models, AI security, Adversarial prompt engineering, Prompt injection, Jailbreak attacks, Cybersecurity education, Virtual labs, Dual-use AI

ACM Reference Format:

Dhanraj Jagadish Devadiga, Ievgeniia Kuzminykh, Hannah Cao, and Bogdan Ghita. 2026. A Virtual Lab for Learning AI Security and Adversarial Prompt Engineering. In *Proceedings of the 31st ACM Conference on Innovation and Technology in Computer Science Education V. 1 (ITiCSE 2026)*, July 10–15, 2026, Madrid, Spain. ACM, New York, NY, USA, 7 pages. <https://doi.org/10.1145/3803400.3809384>

1 Introduction

The rapid integration of Large Language Models (LLMs) into software development and cybersecurity workflows represents a dual challenge for computing education. While these models offer transformative potential for enhancing developer productivity and automating defensive security tasks, their capacity to lower the barrier for exploit creation and sophisticated attacks introduces a novel and critical dimension to the curriculum [5, 18, 23]. Future computing professionals must be equipped not only to use these powerful tools but also to critically assess their security implications and potential for misuse. This paper addresses a pressing gap in computing education by presenting the design and rationale of the Adversarial AI Lab, a virtual learning environment where students gain hands-on, safe experience with the core security concepts of adversarial prompt engineering and AI dual-use risk.

1.1 Research Gap

In the LLM era, the notion of “script-kiddies”, beginner hackers using pre-made tools, has evolved significantly. Research demonstrates that LLMs can now guide users with minimal expertise through complex attacks, generating tailored exploit code and step-by-step instructions [7, 25]. Consequently, the modern computing professional must understand adversarial techniques, such as prompt engineering, where carefully crafted inputs can bypass an LLM’s safety guardrails to generate harmful content or exploit code [2, 19, 33]. Understanding these mechanisms is no longer a niche research topic but a core component of security literacy in the age of AI.

Students in cybersecurity and software engineering programmes are often aware of theoretical risks of generative AI misuse [2, 19, 33], however, they are not provided with structured, ethical opportunities to investigate such adversarial behaviour. Existing



This work is licensed under a Creative Commons Attribution 4.0 International License. *ITiCSE 2026, Madrid, Spain*

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2634-7/2026/07

<https://doi.org/10.1145/3803400.3809384>

educational tools, such as capture-the-flag (CTF) challenges, cyber range, OWASP WebGoat [22] or network penetration labs, excel at teaching traditional vulnerabilities but do not incorporate the dynamic, interactive element of probing AI assistants, as is common in traditional virtual cybersecurity laboratory environments [13]. This creates an educational gap in practical experience: graduates may enter the workforce proficient in using LLMs for code generation, yet remain unprepared for the experience of how the same technology can be weaponised, leaving them ill-equipped to design robust defences or formulate effective AI security policies [17].

1.2 Aim and Scope

To address this gap, we present the Adversarial AI Lab, a virtual learning environment designed to embed students in the investigation of LLM security. The lab enables students to safely engage with the complete investigative cycle: crafting adversarial prompts for different LLMs (e.g., GPT-4, Claude, open-source variants), testing generated outputs against isolated target systems, and analysing results using authentic security metrics. The lab’s design is grounded in expert validation, ensuring students interact with realistic, measurable AI behaviours.

This paper details the lab’s architecture, its scaffolded learning modules covering three primary vulnerability categories: 1) web-based vulnerabilities such as SQL injection, cross-site scripting, and remote code execution, 2) code-based vulnerabilities across multiple programming languages and frameworks, and 3) system-level vulnerabilities including privilege escalation, buffer overflow, memory corruption, and application-specific services. Our work contributes a replicable model for integrating practical AI threat assessment into computing curricula, aiming to develop the critical evaluation skills necessary for responsible innovation in a generative AI technology.

2 Related Work

The dual-use nature of LLMs is well established in security research.

2.1 Offensive Capabilities of LLMs

A robust body of work details the offensive potential of LLMs, providing the factual basis for our educational modules. Studies demonstrate that prompt injection and jailbreaking attacks can bypass built-in safety filters, inducing models to generate harmful content, disclose training data, or produce functional exploit code [23, 24, 30]. The automation of malicious tasks is a key concern; LLMs can assist in vulnerability discovery and exploitation within LLM-integrated applications [15]. Crucially, evaluations reveal measurable variation in model susceptibility. For instance, Pathade et al. [23] quantified differential Attack Success Rates (ASR), showing that models like GPT-4 are highly cooperative, while others, such as Claude, exhibit stronger safety alignment through higher refusal rates. This empirical evidence confirms that LLMs are not uniformly secure and possess distinct, testable risk profiles.

2.2 Defensive Strategies

In parallel, the research community has proposed various defense mechanisms. These include mitigation approaches such as system-mode self-reminders [35], as well as methodological advances in

evaluation. Frameworks for red-teaming and benchmarking have been developed to systematically assess model resilience [9, 16]. Notably, work by Jin et al. [11] and others has highlighted the need for standardised, reproducible evaluation metrics, such as Attack Success Rate and Exploit Generation Accuracy (EGA), to reliably compare model behaviours and defensive efficacy across studies.

2.3 LLMs in Cybersecurity Education

Despite this rich technical discourse, a review of the literature reveals a pronounced gap in educational resources designed for hands-on learning. Many papers discuss safe integration of GenAI in cybersecurity education, demonstrating its potential to enhance critical thinking and awareness [3, 4]. While surveys and taxonomies effectively categorise threats, such as prompt hacking [8, 9, 20], and advanced frameworks demonstrate automated vulnerability exploitation for research or professional penetration testing [15, 21], these materials are not packaged for classroom adoption.

A few studies proposed hands-on adversarial LLM labs. The study [31] explored an approach where students built and attacked a RAG chatbot, learning about data poisoning and retrieval manipulation through direct exploitation. A complementary study [32] presented a structured module where students engaged in jailbreaking activities to exploit model alignment and privacy vulnerabilities, achieving 88% post-module quiz scores and 90% reported confidence.

While these studies confirm that hands-on adversarial labs are feasible and effective, they focus primarily on RAG-specific attacks and open-source models, leaving unexplored the broader domain of LLM exploit generation across diverse vulnerability types (web, code, system) and systematic comparison of commercial versus open-source model behaviours. Our work directly addresses this gap.

3 Lab Design

The lab is built on an isolated virtual network, ensuring no interaction with external systems, in line with established practices for virtual cybersecurity laboratories [14]. The core architecture, illustrated in Figure 1, consists of two key components.

The isolated test network: A Kali Linux virtual machine provides students with standard security testing tools (e.g., curl, Burp Suite). Three target machines, including Metasploitable2, Ubuntu, and a Windows 10 system, offer a diverse landscape of controlled vulnerabilities. This isolated sandbox allows students to execute tests without legal or ethical risk.

The LLM interaction layer provides access to multiple LLMs, including commercial APIs (GPT-4, Claude, Copilot, Gemini) and a locally hosted instance of LLaMA, enabling comparative analysis. The entire environment can be reset to a clean snapshot, ensuring repeatability and a consistent starting point for all students.

3.1 The Student Learning Workflow

The lab guides students through a structured, human-in-the-loop workflow mirroring a professional security assessment, as shown in Figure 2.

The “Human Tester” in the diagram represents the student, who crafts adversarial prompts, submits them to the LLM layer, and

then manually executes any generated payload within the isolated test network. This clear separation between the AI interaction and the execution environment is a fundamental safety and pedagogical feature, teaching students the critical practice of validating AI output in a controlled space.

Students analyse outcomes using a logging framework and calculate key performance metrics to enable quantitative comparison. This transitions them from observation to evidence-based assessment, aligning with prior AI-enabled educational platforms that

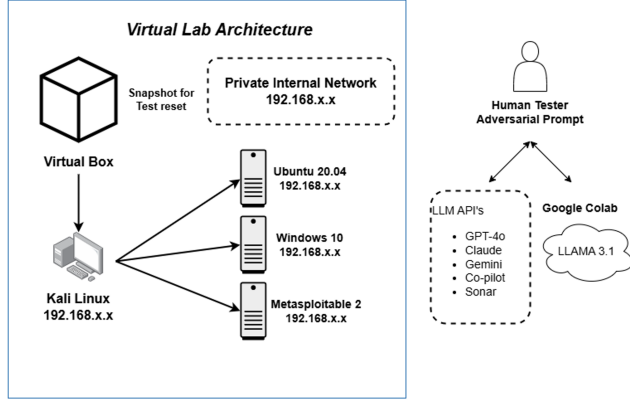


Figure 1: The design architecture for testing the adversarial prompts using LLMs

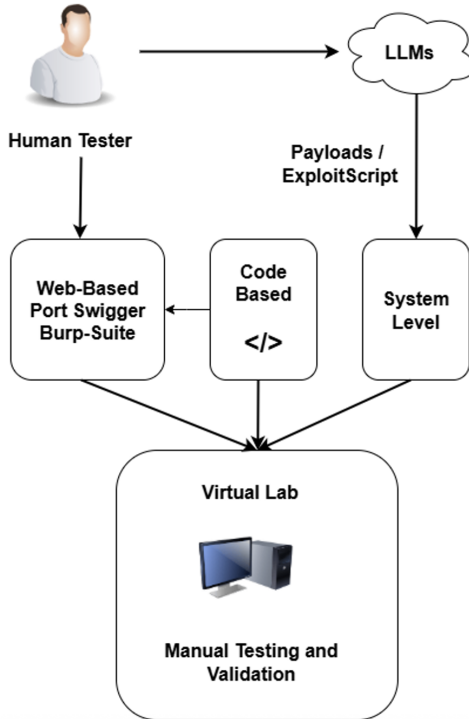


Figure 2: Testing setup for scaffolded learning modules

use automated feedback and adaptive analysis to support reflective learning and skill development [29]. This transitions them from observation to evidence-based assessment. The core metrics, adapted from research frameworks [11, 23], are:

Attack Success Rate (ASR): Measures a model's ability to generate working exploits.

$$ASR = \frac{\text{Number of successful exploits}}{\text{Total number of exploit attempts}} \times 100$$

Exploit Generation Accuracy (EGA): Assesses the syntactic and semantic correctness of the code.

$$EGA = \frac{\text{Number of technically valid exploit payloads}}{\text{Total generated payloads}} \times 100$$

Refusal Rate: Quantifies how often a LLM declined to generate exploit-related content due to safety or policy restrictions.

$$\text{Refusal Rate} = \frac{\text{Number of refused responses}}{\text{Total prompts}} \times 100$$

Vulnerability Exploit Success (VES): A qualitative score (1-6 for code-based, 1-3 for web-based) reflecting the overall quality, contextual relevance, and feasibility of each exploit. The metrics for the score are shown in Tables 1 and 2.

$$VES = \frac{\sum_{i=1}^n VES_i}{n}$$

where VES_i is the vulnerability explanation score for the i -th prompt, and n is the total number of prompts evaluated for the model. A high VES score means the model identifies vulnerabilities and provides a detailed explanation. On the contrary, a low VES score suggests that the model does not provide proper reasoning and is limited by safety restrictions.

Mitigation Steps Provided (MSP): Measured the proportion of responses in which the model offered meaningful defensive or remediation guidance after identifying a vulnerability.

$$MSP = \frac{\text{Number of responses with secured code}}{\text{Total prompts}} \times 100$$

3.2 Scaffolded Learning Modules

The testing setup consists of three modules, each focusing on a different vulnerability layer and incorporating tiered difficulty levels (*Apprentice*, *Practitioner*, and *Expert*), reflecting established instructional scaffolding principles [28] and recent approaches that employ Generative AI to support progressive learning in cybersecurity education [34].

Module 1: Web-Based Vulnerability Testing. This module focuses on threats such as SQL Injection (SQLi) and Cross-Site Scripting (XSS) from the OWASP Top 10. Students first use descriptive prompts requesting general explanations, which often yield limited exploitable output. They then advance to contextual prompting by providing vulnerable scripts directly, asking the model to identify, explain, and exploit the flaw. Students test the generated payloads against the PortSwigger platform, learning the critical impact of prompt specificity and score as in Table 1.

Exercises are tiered across three difficulty levels, Apprentice, Practitioner, and Expert, adapted from the PortSwigger Web Security Academy [21, 26, 27]. This scaffolding supports differentiated learning and allows students to observe how prompt complexity and

Table 1: Web-based testing scoring scale

Score	Reason
1	No explanation was given, or the model refused to respond
2	The model gave a brief or incomplete explanation with minimal technical context
3	The model offered a complete and detailed explanation with proper technical reasoning

model performance interact, while intentionally surfacing forms of troublesome knowledge that require learners to reassess incorrect assumptions before progressing [12].

Module 2: Code-Based Vulnerability Testing. Here, students assess LLMs’ ability to reason about code-level flaws, such as buffer overflows, across multiple programming languages. The objective is to evaluate not only exploit generation but also the model’s propensity to “hallucinate” vulnerabilities in secure code. Students are given code snippets, some vulnerable, some not, and analyse the accuracy and technical precision of the LLM’s output. This module reinforces that code generation capability does not equate to correct security analysis. Evaluation focuses on EGA, ASR, and the detailed code-based VES scale (Table 2), which rewards models that provide both exploits and secure patches.

Table 2: Code-based testing scoring scale

Score	Reason
1	The model refused or failed to respond
2	The model described the vulnerability but provided neither a payload nor secure code
3	The model explained the vulnerability and included secure code but no payload
4	The vulnerability and a payload were described, but the exploit failed, and no secure code was given
5	The explanation and payload were present, the exploit may or may not have worked, and secure code was not provided
6	The model gave a clear explanation, generated a successful payload, and included a secure version of the code

Module 3: System-Level Vulnerability Testing. Students evaluate how LLMs can support red-teaming processes against entire systems. Using the diverse target VMs (Metasploitable2, configured Ubuntu, Windows 10, as shown in Figure 1), they provide models with system information (e.g., open ports, services) and prompt for attack chains, privilege escalation vectors, or payload generation. Success is evaluated based on situational suitability (whether the proposed exploit is applicable to the specific target configuration) teaching students to contextualise AI-generated attack plans. ASR and qualitative notes are the primary tools for analysis in this case.

Table 3: Web-based vulnerability module validation

Model	ASR (%)	EGA (%)	ART (seconds)	VES (1-3)	Refusal Rate (%)
GPT-4o	77.5	67.5	30.62	2.63	0
Claude	43.02	39.53	30.88	1.45	32.56
Copilot	70	62.5	24.10	2.11	0
Gemini	68.75	52.5	23.85	1.97	0
Sonar	83.75	75	29.90	2.60	0

3.3 Reporting and Assessment Rubric

To focus student inquiry and enable assessment, the lab is supported by two core elements.

Written Report: For each module, students complete a worksheet prompting them to 1) record their initial descriptive prompt and the LLM’s response; 2) record their refined contextual prompt and the generated payload; 3) document the manual test result (Success/Fail/Refusal); 4) calculate a simple metric (e.g., ASR for their 3-5 tests); and 5) write a brief comparative analysis.

Assessment Rubric: A final synthesis report is evaluated on criteria including the use of evidence (data from the report, depth of comparative analysis of LLMs), application of security metrics (correct use of ASR, Refusal Rate), and ethical reasoning (discussing dual-use implications).

This design ensures that the technical complexity of adversarial AI research is made accessible, transforming students from passive learners into active analysts capable of empirically evaluating the security implications of intelligent systems.

4 Evaluation

We performed expert validation of the lab’s infrastructure prior to student deployment using the metrics presented in Section 3.1. This validation involved three cybersecurity educators, each with over 15 years of teaching and industry experience, who systematically tested the lab environment against the defined scenarios.

4.1 Web-Based Vulnerability Evaluation

Our evaluation of the OWASP Top 10 vulnerabilities revealed clear differences in model profiles (Table 3). Sonar and GPT-4o exhibited the highest Attack Success Rates of 83.75% and 77.5%, respectively, demonstrating a strong ability to generate functional web exploits. In contrast, Claude showed the highest safety alignment, with a Refusal Rate of 32.56% and the lowest ASR (43.02%). This quantifiable spectrum, from high “cooperativeness” to high caution, provides students with a concrete basis for comparative analysis.

Furthermore, the tiered analysis provided deeper pedagogical insights. While most models performed consistently, Claude showed an increased ASR at the Expert level (see Figure 3). This result demonstrates that even safety aligned LLMs can be bypassed with sophisticated prompting, a key concept for students to explore [23].

Analysis of Average Response Time across levels of difficulties (Figure 4) showed that models like GPT-4o and Claude maintained consistent speed regardless of difficulty, while others (e.g., Copilot, Gemini) showed variable performance. This teaches students to consider not just whether a model can solve a problem, but also its

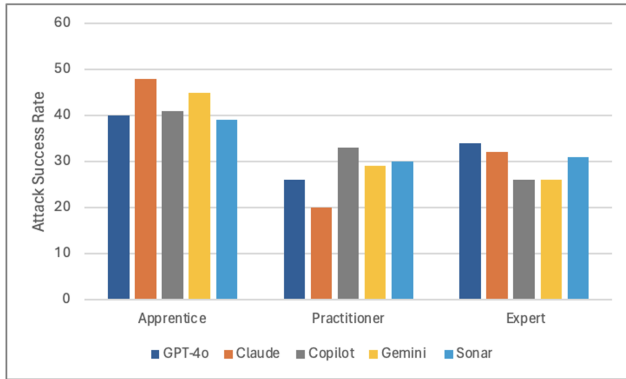


Figure 3: Web-based evaluation result for Attack Success Rate across expertise levels

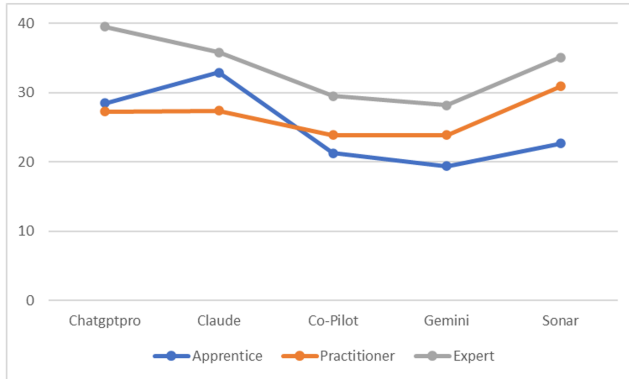


Figure 4: Average Response Time for each LLM across three task difficulty levels (in seconds)

resource efficiency and the reliability of its reasoning under varying cognitive loads.

4.2 Code-Based Vulnerability Evaluation

Testing on code snippets highlighted risks beyond mere exploit generation. As shown in Table 4, while GPT-4o again led in ASR (65%) and Exploit Generation Accuracy (75%), other models presented different risk profiles. Copilot achieved the highest Vulnerability Exploit Success score (3.88), indicating its successful payloads were highly detailed and impactful. Gemini had a notably low Mitigation Steps Provided (MSP) score (15%), suggesting its outputs might be stealthier and less likely to raise defensive alarms [23]. Claude and LLaMA 3.1 performed poorly on exploit metrics, emphasizing the security-performance trade-off in open-source and tightly aligned models. These results allow students to investigate not only whether an LLM generates code, but also its quality, completeness, and ethical alignment of that code by the MSP metric.

4.3 System-Based Vulnerability Evaluation

The qualitative evaluation against full system targets (Metasploitable2, Ubuntu, Windows 10) tested LLMs' ability to reason about context.

Table 4: Code-based vulnerability module validation

Model	ASR (%)	EGA (%)	ART (min)	VES (1-5)	MSP (%)
GPT-4o	65	75	11	3.69	52.5
Claude	11.25	11.25	17.5	1.75	52.5
Copilot	53.75	53.75	17.5	3.88	46.25
Gemini	57.5	61.25	20	3.39	15
LLaMA 3.1	22.5	30	22.5	1.67	40

Table 5: System-based vulnerability module validation

Model	Metasploitable 2	Ubuntu 20.04	Windows 10
GPT-4o	Full chain (RCE + root)	Working with minor fixes	Suggested obfuscated payloads, all blocked
Claude	Partial output with disclaimer	Refused	Refused
Copilot	Educational only exploit	Worked, limited depth	Outdated suggestions
Gemini	Refused	Limited with safety notice	Basic hints only
LLaMA 3.1	Ineffective script	Refused	Refused

GPT-4o demonstrated the greatest adaptability, knowing how to generate full attack chains for Linux targets and suggesting obfuscated payloads for a hardened Windows environment (though they were blocked by defences). Other models (Claude, Gemini, LLaMA) frequently refused or provided only partial, educational outputs (Table 5). This stark contrast allows students to examine when and why LLMs refuse to cooperate, and how their utility varies dramatically depending on the target environment and prompt specificity.

4.4 Prompt Transferability across Models

A critical finding for student discussion is the high transferability of successful adversarial prompts across different model architectures. Prompts that jailbroke one model often compromised others, with GPT-4o and Gemini being particularly susceptible. This demonstrates that vulnerabilities can be architectural rather than model-specific, a fundamental lesson in AI security [23].

5 Discussion

The Adversarial AI Lab was designed to address a specific and growing need in computing education: providing safe, hands-on experience with the dual-use security implications of generative AI. Our expert validation confirms that LLM behaviour can be empirically characterised using established security metrics such as attack success rate and refusal rate, as demonstrated in prior evaluations of adversarial prompting and automated exploitation [8, 23]. Our results show GPT-4o frequently complies with requests to generate exploits (high Attack Success Rate), and Claude frequently refuses such requests [15, 23]. This section discusses the lab's pedagogical benefits, practical considerations for adoption, and limitations.

5.1 Pedagogical Benefits

While a formal study with students is planned, the lab provides a framework for responsible education. It provides a controlled venue where the provocative question “How can this be misused?” can be investigated safely, turning a potential source of harm into a foundation for developing responsible, ethically-aware computing professionals.

A key social concern is that LLMs can empower AI-assisted script kiddies [5, 23], amplifying an already documented mismatch between the skills taught in higher education and the competencies demanded by industry in the AI and cybersecurity sectors [10]. The lab flips this risk into a teaching moment. Providing a safe sandbox allows students to empirically observe this capability, thereby deepening their understanding of the threat landscape they must defend against. Furthermore, exercises on model hallucination (e.g., generating exploits for non-existent vulnerabilities) teach critical evaluation skills, training students to never blindly trust AI output, a crucial defence against misinformation [11, 23].

By using simplified versions of research metrics (ASR, Refusal Rate, MSP), students learn to move from subjective opinion (“Model X is riskier”) to evidence-based argument (“Model X has a 65% ASR and a 15% MSP, indicating high exploit success but low ethical alignment”) [23].

The lab forces engagement with the core dual-use dilemma. Students engage with the same tensions professionals face: the same prompt that can generate a security patch can also generate an exploit [6, 21]. The Mitigation Steps Provided (MSP) metric, for instance, directly frames the professional responsibility to not only identify flaws but also fix them, reflecting established norms in responsible vulnerability disclosure and secure software engineering practice [1, 6]. The guided worksheets and final report rubric explicitly require students to reflect on this duality, the societal risks of lowering barriers to cybercrime, and their professional responsibilities as outlined by professional bodies like the British Computer Society [1]. This moves learning beyond technical skill to encompass ethical reasoning and professional integrity.

5.2 Challenges, and Limitations

Adopting the lab requires consideration of resources and facilitation. Firstly, the technical setup requires dedicated institutional resources. Instructors and students need access to capable hardware for running multiple virtual machines and, optionally, API credits for commercial LLMs. While using open-source models like LLaMA and free tiers can mitigate costs, the initial configuration of the isolated network and virtual machines requires a moderate level of technical proficiency from the instructor.

Secondly, the lab captures a snapshot of a rapidly evolving landscape. The performance, safety filters, and even the availability of the evaluated LLMs (e.g., GPT-4o, Claude) are subject to change by their developers. Consequently, the specific numerical results from our expert validation (e.g., GPT-4o’s 77.5% ASR) serve as an illustrative baseline rather than a permanent answer key. The core pedagogical value, however, remains in the comparative framework as the students will still discover meaningful differences in model behaviour, even if the absolute metrics shift over time.

Finally, the lab’s scope is intentionally focused on technical exploit generation to maintain a clear learning objective. It does not extensively cover other critical dimensions of AI security, such as data poisoning attacks, membership inference, or the generation of convincing phishing and disinformation content. These areas represent valuable avenues for expanding the curriculum in future iterations of the lab or as separate advanced modules.

5.3 Future Work

The immediate next step is the formal educational study outlined in this paper’s introduction, which will measure the lab’s impact on students’ learning, threat-modelling skills, and ethical reasoning. Furthermore, the architecture supports advanced exploration. The concept of autonomous red-teaming agents, hinted at in recent research on LLM-driven autonomous exploitation [8, 20], could form the basis for a graduate-level module where students architect LLM-powered agents within the safe lab confines, exploring the cutting edge of AI security research responsibly.

6 Conclusions

The integration of Large Language Models into the software development lifecycle is not a future possibility but a present reality, bringing with it profound dual-use security challenges. Computing education has a responsibility to prepare graduates who are not only proficient users of these tools but also discerning critics of their capabilities and risks. This paper presented the Adversarial AI Lab, a virtual learning environment that meets this responsibility by transforming security research into a safe, structured, and investigatory educational experience.

We detailed the lab’s isolated architecture, its scaffolded modules across web, code, and system-level vulnerabilities, and its pedagogical framework that uses authentic performance metrics to guide student analysis. Crucially, we grounded its design in expert validation, demonstrating that it exposes students to the same measurable LLM behaviours, variations in cooperativeness, safety alignment, and ethical output that define the current research frontier [11, 15, 23]. By requiring manual validation and incorporating ethical reflection, the lab models the professional diligence required to navigate the dual-use landscape responsibly.

References

- [1] British Computer Society. 2021. BCS Code of Conduct. <https://www.bcs.org/membership-and-registrations/become-a-member/bcs-code-of-conduct/>. Accessed: July 21, 2025.
- [2] Nicholas Carlini, Florian Tramèr, Eric Wallace, Matthew Jagielski, Ariel Herbert-Voss, Katherine Lee, Adam Roberts, Tom Brown, Dawn Song, Úlfar Erlingsson, Alina Oprea, and Colin Raffel. 2021. Extracting Training Data from Large Language Models. In *USENIX Security Symposium*. 2633–2650.
- [3] Anjali Chaudhary, Areej Javed, Walter Colombo, and Fernando A. Manchego. 2025. Exploring the Safe Integration of Generative AI in Cybersecurity Education: Addressing Challenges in Transparency, Accuracy, and Security. In *Advances in Teaching and Learning for Cyber Security Education*, Phil Legg, Natalie Coull, and Charles Clarke (Eds.). Lecture Notes in Networks and Systems, Vol. 1213. Springer, Cham. doi:10.1007/978-3-031-77524-6_1
- [4] Mahmoud Elkhodr and Ergun Gide. 2025. Integrating Generative AI in Cybersecurity Education: Case Study Insights on Pedagogical Strategies, Critical Thinking, and Responsible AI Use. arXiv:2502.15357. doi:10.48550/arXiv.2502.15357
- [5] Mohamed Amine Ferrag, Fatima Alwahedi, Ammar Battah, Bilel Cherif, Abdechakour Mechri, Norbert Tihanyi, Tamas Bisztray, and Merouane Debbah. 2025. Generative AI in Cybersecurity: A Comprehensive Review of LLM Applications and Vulnerabilities. *Internet of Things and Cyber-Physical Systems* 5 (2025), 1–46. doi:10.1016/j.iotcps.2025.01.001

- [6] Alexei Grinbaum and Laurynas Adomaitis. 2024. Dual Use Concerns of Generative AI and Large Language Models. *Journal of Responsible Innovation* 11, 1 (2024). doi:10.1080/23299460.2024.2304381
- [7] Walid Hariiri. 2023. Unlocking the Potential of ChatGPT. arXiv:2304.02017. <https://arxiv.org/abs/2304.02017>
- [8] Julius Henke. 2025. AutoPentest: Enhancing Vulnerability Management with Autonomous LLM Agents. *arXiv preprint arXiv:2505.10321* (2025). <https://arxiv.org/abs/2505.10321> arXiv:2505.10321.
- [9] Junjie Huang and Quanyan Zhu. 2024. PenHeal: A Two-Stage LLM Framework for Automated Pentesting and Optimal Remediation. In *Proceedings of the Workshop on Autonomous Cybersecurity (AutonomousCyber '24)*. ACM, 11–22. doi:10.1145/3689933.3690831
- [10] Khushi Jaiswal, Ievgeniia Kuzminykh, and Sanjay Modgil. 2024. Understanding the skills gap between higher education and industry in the UK artificial intelligence sector. *Industry and Higher Education* 39, 2 (2024), 234–246. doi:10.1177/09504222241280441
- [11] David Jin, Qian Fu, and Yuekang Li. 2025. Good News for Script Kiddies? Evaluating Large Language Models for Automated Exploit Generation. In *IEEE Security and Privacy Workshops (SPW)*. IEEE, 278–282. doi:10.1109/SPW67851.2025.00039
- [12] Ievgeniia Kuzminykh, Bogdan Ghita, Hannan Xiao, Mykola Yevdokymenko, and Oleksandr Yermenko. 2021. Investigating Threshold Concept and Troublesome Knowledge in Cyber Security. In *2021 1st Conference on Online Teaching for Mobile Education (OT4ME)*. Alcalá de Henares, Spain, 26–30. doi:10.1109/OT4ME53559.2021.9638889
- [13] Oleksandr Lemeshko, Oleksandr Yermenko, Mykola Yevdokymenko, and Ievgeniia Kuzminykh. 2020. Features of Developing a Virtual Cybersecurity Laboratory for Distance Learning. *New Collegium* 3, 101 (2020), 41–45.
- [14] Oleksandr Lemeshko, Oleksandr Yermenko, Mykola Yevdokymenko, Ievgeniia Kuzminykh, and Abdelhak Mersni. 2021. Development of Virtual Laboratories and Innovative Cybersecurity Courses for Distance Learning. In *CEUR Workshop Proceedings*, Vol. 3188. 98–107.
- [15] Tong Liu, Zizhuang Deng, Guozhu Meng, Yuekang Li, and Kai Chen. 2024. Demystifying RCE Vulnerabilities in LLM-Integrated Apps. In *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security (CCS '24)*. ACM, 1716–1730. doi:10.1145/3658644.3690338
- [16] Yupei Liu, Yuqi Jia, Runkeng Geng, Jinyuan Jia, and Neil Zhenqiang Gong. 2024. Formalizing and Benchmarking Prompt Injection Attacks and Defenses. In *Proceedings of the 33rd USENIX Security Symposium (USENIX Security 2024)*. USENIX Association, 1831–1847. <https://www.usenix.org/conference/usenixsecurity24/presentation/liu-yupei>
- [17] Eric Burton Martin and Sudipto Ghosh. 2023. GitHub Copilot: A Threat to High School Security?. In *International Conference on Emerging Trends in Networks and Computer Communications*. IEEE. doi:10.1109/ETNCC59188.2023.10284976
- [18] Shivani Metta, Isaac Chang, Jack Parker, Michael P. Roman, and Arturo F. Ehuán. 2024. Generative AI in Cybersecurity. Coalfire White Paper. <https://coalfire.com/insights/resources/white-papers/generative-ai-in-cybersecurity>
- [19] Farzad Nourmohammadzadeh Motlagh, Mehrdad Hajizadeh, Mehryar Majd, Pejman Najafi, Feng Cheng, and Christoph Meinel. 2024. Large Language Models in Cybersecurity: State-of-the-Art. arXiv:2402.00891. <https://arxiv.org/abs/2402.00891>
- [20] Sho Nakatani. 2025. RapidPen: Fully Automated IP-to-Shell Penetration Testing with LLM-Based Agents. *arXiv preprint arXiv:2502.16730* (2025). <https://arxiv.org/abs/2502.16730> arXiv:2502.16730.
- [21] OWASP Foundation. 2025. OWASP Top Ten. <https://owasp.org/www-project-top-ten/> Accessed: July 21, 2025.
- [22] OWASP Foundation. 2025. OWASP WebGoat Project. <https://owasp.org/www-project-webgoat/> Accessed: July 21, 2025.
- [23] Chetan Pathade. 2025. Red Teaming the Mind of the Machine: A Systematic Evaluation of Prompt Injection and Jailbreak Vulnerabilities in LLMs. arXiv:2505.04806. <https://arxiv.org/abs/2505.04806>
- [24] Fábio Perez and Ian Ribeiro. 2022. Ignore Previous Prompt: Attack Techniques for Language Models. arXiv:2211.09527. doi:10.48550/arXiv.2211.09527
- [25] Duy Anh Pham, Ievgeniia Kuzminykh, Andrii Astrakhantsev, and Bogdan Ghita. 2025. Evaluating Large Language Models for Anomaly Detection with Cost-Efficient Sampling: A Generalized Framework. In *IEEE International Black Sea Conference on Communications and Networking (BlackSeaCom)*. Chisinau, Moldova.
- [26] PortSwigger. 2025. All Web Security Academy Topics. <https://portswigger.net/web-security/all-topics> Accessed: July 21, 2025.
- [27] PortSwigger. 2025. Web Application Security Testing Software – Burp Suite. <https://portswigger.net/burp/application-security-testing> Accessed: July 21, 2025.
- [28] Brian J. Reiser. 2004. Scaffolding Complex Learning: The Mechanisms of Structuring and Problematising Student Work. *Journal of the Learning Sciences* 13, 3 (2004), 273–304. doi:10.1207/s15327809jls1303_2
- [29] Ugur Tepe, Ievgeniia Kuzminykh, Hannan Xiao, and Maher Salem. 2025. AI-Enabled Adaptive Learning Platform CryptoQuiz for Learning Cybersecurity. In *Advances in Cyber Security Education 2025* (2025-07-22). Coventry, UK.
- [30] Eric Wallace, Pedro Rodriguez, Shi Feng, Ikuya Yamada, and Jordan Boyd-Graber. 2019. Trick Me If You Can: Human-in-the-Loop Generation of Adversarial Examples for Question Answering. *Transactions of the Association for Computational Linguistics* 7 (2019), 387–401.
- [31] Dominic Wilson. 2025. A Hands-on Approach to Enhancing LLM Security Education through Retrieval-Augmented Generation. *Journal of Computing Sciences in Colleges* 41, 4 (2025), 173–182. <https://dl.acm.org/doi/10.5555/3787712.3787744>
- [32] Dominic A. Wilson. 2026. Integrating Adversarial Scenarios into LLM Security Labs: An Experience Report on a Hands-On Approach. *Journal of Cybersecurity Education, Research and Practice* 2026, 1 (2026). doi:10.62915/2472-2707.1268
- [33] Xiaodong Wu, Ran Duan, and Jianbing Ni. 2024. Unveiling Security, Privacy, and Ethical Concerns of ChatGPT. *Journal of Information and Intelligence* 2, 2 (2024), 102–115. doi:10.1016/j.jiixd.2023.10.007
- [34] Hannan Xiao, Bhavesh Shah, Jonathan Spring, Ievgeniia Kuzminykh, and Stanislav Janku. 2025. Scaffolding Student Learning through GenAI in Cybersecurity Education. In *Proceedings of the 3rd International Workshop on Cyber Security Education for Industry and Academia (CSE4IA 2025)*. Munich, Germany, 16–18.
- [35] Yueqi Xie, Jingwei Yi, Jiawei Shao, Justin Curl, Lingjuan Lyu, Qifeng Chen, Xing Xie, and Fangzhao Wu. 2023. Defending ChatGPT Against Jailbreak Attack via Self-Reminders. *Nature Machine Intelligence* 5 (2023), 1486–1496. doi:10.1038/s42256-023-00765-8